

به نام خدا

گزارش تمرین اول درس مبانی رایانش توزیع شده

اعضای تیم:

سروش اصفهانیان 810101376

نوید هاشمی 810101549

فهرست مطالب

3.....	بخش اول: ارتباط بین دو پردازش مستقل با Pipe
3.....	معماری و نحوه طراحی
3.....	پروتکل ارتباطی میان Worker و Interface
4.....	اجرا:
5.....	بخش دوم: همزمانی، زمان بندی و مشاهده اثرات Context-Switch
5.....	پیاده سازی:
7.....	اجرا:
8.....	تحلیل نتایج:
13.....	پاسخ به سوالات:
14.....	بخش سوم: ساخت سرویس، کانتینر سازی و اجرا در ماشین مجازی

بخش اول: ارتباط بین دو پردازش مستقل با Pipe

این قسمت شامل پیاده سازی یک سیستم ساده اما قابل اتکا برای اجرای محاسبات به صورت توزیع شده است. هدف اصلی جدا کردن مسئولیت ها میان دو جزء مستقل بود: پردازش محاسبات در یک **process** به نام **Worker** و مدیریت ورودی/خروجی و تعامل با کاربر در یک **process** دیگر به نام **Interface**. این جداسازی باعث می شود خطاهای لایه کاربری روی بخش محاسبات اثر نگذارد و در عین حال امکان توسعه و اشکال زدایی هر بخش نیز ساده تر باشد.

معماری و نحوه طراحی

ساختار سیستم از دو برنامه ی مستقل تشکیل شده است:

1. Worker

این بخش مسئول انجام عملیات ریاضی و تولید خروجی در قالب JSON است. **Worker** پس از اجرا دو **named pipe** ایجاد می کند؛ یکی برای دریافت درخواست ها و دیگری برای ارسال پاسخ. این فرآیند تا زمانی فعال می ماند که ورودی معتبر از **Interface** دریافت کند و در صورت دریافت سیگنال خروج یا بروز خطای جدی، **pipe** ها را به طور کامل حذف می کند تا از باقی ماندن فایل های بدون استفاده جلوگیری شود.

2. Interface

این بخش تنها با کاربر تعامل دارد. تمام ورودی ها پیش از ارسال به **Worker** مورد بررسی قرار می گیرند (به نوعی بخشی از بررسی های ابتدایی در سمت **client** قرار گرفته) تا از ارسال داده ی نادرست جلوگیری شود. این لایه همچنین تمامی خطاهای احتمالی سمت کاربر و خطاهای ناشی از ارتباط **pipe** را مدیریت کرده و پیام های مناسب نمایش می دهد.

در طراحی این بخش تأکید بر این بوده که هیچ ورودی ناسازگار، دستور ناشناس یا آرگومان غیر عددی از مرز **pipe** عبور نکند. این جداسازی منجر به کوچک تر شدن سطح خطا در **Worker** و رفتار قابل پیش بینی تر سیستم شده است.

این معماری دولایه باعث استقلال عملکردی دو بخش شده و منطق پردازش را کاملاً از مدیریت ورودی ها جدا کرده است. همچنین، استفاده از IPC مبتنی بر **pipe** باعث حذف وابستگی به شبکه و ساده تر شدن مدل ارتباطی می شود.

پروتکل ارتباطی میان Worker و Interface

همانطور که گفته شد، ارتباط میان دو **process** یاد شده تنها از طریق دو **named pipe** در مسیرهای **tmp/dist_req.pipe/** و **tmp/dist_res.pipe/** انجام می شود. این پروتکل کاملاً متنی است و مراحل آن به صورت زیر تعریف شده:

1. ارسال درخواست: **Interface** پس از اعتبارسنجی ورودی، یک رشته متنی با ساختار زیر تولید کرده و از طریق **pipe** ورودی ارسال می کند:

COMMAND arg1 arg2

در این ساختار، **COMMAND** یکی از عملیات پشتیبانی شده است و **arg1** و **arg2** مقادیر عددی هستند.

2. پاسخ: **Worker** نتیجه ی پردازش را همیشه در قالب JSON برمی گرداند. فرمت کلی پاسخ به صورت زیر است:

{"result": <value>, "error": "<error_message_or_empty>"}

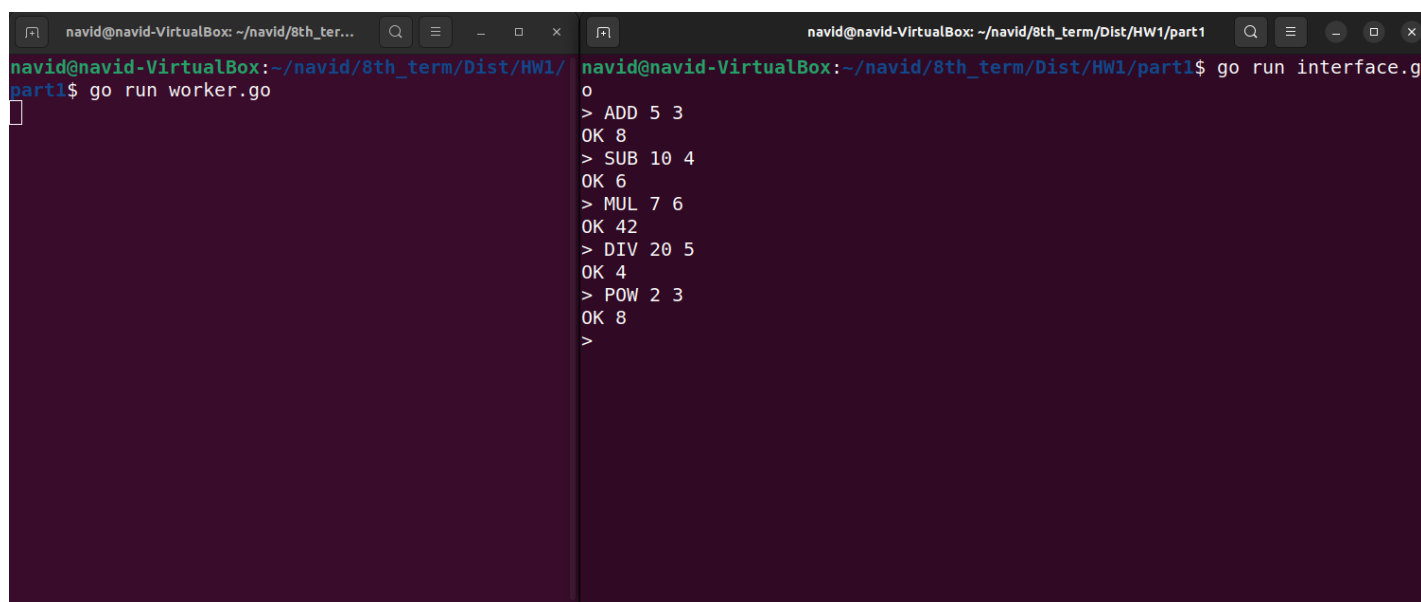
در صورت موفقیت، فیلد **error** خالی است و در صورت بروز خطا، مقدار **result** نامعتبر خواهد بود و پیام خطا در **error** قرار می‌گیرد. استفاده از JSON این امکان را می‌دهد که **Interface** بدون وابستگی به نحوه‌ی چاپ یا فاصله‌گذاری، به‌صورت ساختاریافته پاسخ را تجزیه کند و پیام مناسب برای کاربر نمایش دهد.

3. در صورت بسته شدن ناگهانی **worker**، **pipe** خروجی بدون داده باقی می‌ماند و **Interface** آن را به عنوان قطع ارتباط تشخیص می‌دهد. در واقع اگر **pipe** پاسخ‌دهی نداشته باشد یا داده ناقص باشد، خطای **pipe_read_error** ثبت می‌شود و در صورت نبودن **pipe** ها هنگام شروع برنامه، **Interface** نتیجه می‌گیرد که **worker** اجرا نشده و با پیام مناسب متوقف می‌شود.

این پروتکل، حداقلی، پایدار و قابل تشخیص است و تقریباً تمام سناریوهای خطا را چه در سمت **Worker** و چه در سمت **Interface** پوشش می‌دهد.

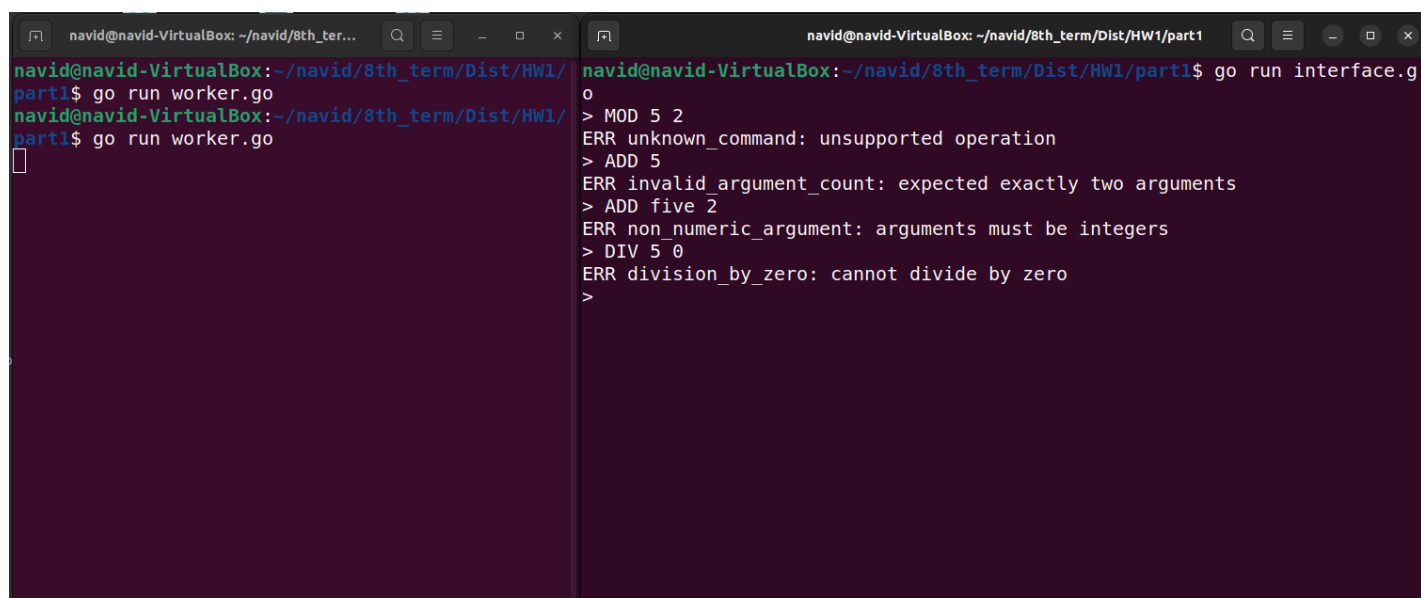
اجرا:

در ادامه این قسمت اسکرین شات‌هایی از اجرای این دو پردازش آورده‌ام که توضیحات هر تصویر زیر آن آمده است:



```
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW1/part1$ go run worker.go
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW1/part1$ go run interface.g
0
> ADD 5 3
OK 8
> SUB 10 4
OK 6
> MUL 7 6
OK 42
> DIV 20 5
OK 4
> POW 2 3
OK 8
>
```

Figure 1 - نمونه اجرای صحیح برنامه و دریافت پاسخ محاسباتی از Worker



```
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW1/part1$ go run worker.go
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW1/part1$ go run interface.g
0
> MOD 5 2
ERR unknown_command: unsupported operation
> ADD 5
ERR invalid_argument_count: expected exactly two arguments
> ADD five 2
ERR non_numeric_argument: arguments must be integers
> DIV 5 0
ERR division_by_zero: cannot divide by zero
>
```

Figure 2 - اجرای دستورات نادرست برای بررسی عملکرد مدیریت خطا

بخش دوم: همزمانی، زمان‌بندی و مشاهده اثرات Context-Switch

در این بخش، به بررسی نحوه پیاده‌سازی کد مربوط به بخش دوم پروژه و سپس تحلیل نتایج آن و پاسخ به سوالات مطرح‌شده می‌پردازیم. هر کدام از این موارد در ادامه گزارش قابل مشاهده است:

پیاده‌سازی:

کد مربوط به این بخش در فایل `main.go` قابل مشاهده است. در ابتدا و پس از ایمپورت کتابخانه‌های مورد نیاز و تعریف داده‌ساختار متریک که شامل معیارهای مورد اندازه‌گیری می‌باشد، دو تابع `cpuHeavyTask` و `mixedTask` پیاده‌سازی شده‌اند که به ترتیب مربوط به لود کاری CPU-Bound و ترکیبی می‌باشد.

برای تسک CPU-Bound، یک حلقه با تعداد تکرار گرفته شده از ورودی در نظر گرفته شده است که در هر تکرار، متغیر مربوط به حاصل جمع را با حاصل ضرب شمارنده تکرار حلقه در عدد دلخواه 2342 به پیمانه عدد دلخواه دیگری (71) جمع می‌کند، که از آن جایی که تعداد تکرار حلقه به طور پیش‌فرض عددی بزرگ در نظر گرفته می‌شود، انتظار می‌رود این عملیات، پردازنده را به مقدار زیادی درگیر کند و رفتار یک تسک CPU-Bound را از خود بروز دهد.

برای تسک ترکیبی نیز، حالت محاسبه + `sleep` در نظر گرفته شده است، که در تابع مربوطه، ابتدا تسک CPU-Bound با نصف تعداد تکرار صدا زده شده و سپس تابع به مقدار 20 میلی‌ثانیه به `sleep` می‌رود و سپس مجدداً تسک CPU-Bound با نصف تعداد تکرار صدا زده می‌شود. به علت شامل بودن تسک CPU-Bound و همچنین انجام عملیات `sleep` که می‌تواند مانند منتظر ایستادن برای یک عملیات IO باشد، انتظار می‌رود رفتار این تابع نیز مانند حالت لود کاری ترکیبی باشد. در زیر تصویر هر دو تابع توضیح داده‌شده مشاهده می‌شود.

```
// CPU-bound task
func cpuHeavyTask(iter int) {
    sum := 0
    for i := 0; i < iter; i++ {
        sum += (i * 2342) % 71
    }
    _ = sum
}

// Mixed workload task, compute + a sleep
func mixedTask(iter int) {
    cpuHeavyTask(iter / 2)
    time.Sleep(20 * time.Millisecond)
    cpuHeavyTask(iter / 2)
}
```

در ادامه کد، پیاده‌سازی تابع `runExperiment` قرار دارد که با گرفتن مقادیر مربوط به تعداد `goroutine`ها، `GOMAXPROCS` و نوع لود کاری، پس از هم‌گام‌سازی زمان شروع برای روتین‌ها، در یک حلقه، به تعداد `goroutine`ها یک روتین جدید ساخته و بسته به نوع لود کاری مشخص‌شده، تابع مربوطه را در `goroutine` ساخته‌شده صدا می‌زند. در انتهای حلقه، مقدار تأخیر روتین حساب شده و سپس پس از پایان اجرای حلقه، تابع `Wait` برای اطمینان از اتمام تمامی روتین‌ها صدا زده شده و در انتها مقدار تأخیر کل ذخیره می‌شود تا بدین ترتیب علاوه بر تأخیر هر تکرار حلقه، تأخیر کل برنامه را هم داشته باشیم.

در ادامه تابع، با استفاده از مقادیر تأخیر ذخیره‌شده، معیارهای مربوطه شامل کمترین تأخیر، بیشترین تأخیر، انحراف معیار، `throughput` و... محاسبه شده و داده‌ساختار `Metrics` ساخته شده بازگردانی می‌شود. لازم به ذکر است که برای محاسبه تعداد دفعات بلاک شدن برنامه، پس از خروج از اجرای تابع `mixedTask` در روتین، مقدار متغیر `blockingCount` به صورت اتمیک افزایش پیدا می‌کند تا در محاسبه این مقدار، `Mutual Exclusion` رعایت شده و مقدار درستی در انتها گزارش شود.

تابع `printMetrics` یک نمونه از داده‌ساختار `Metrics` را دریافت کرده و مقادیر آن را به شکلی خوانا چاپ می‌کند. پیاده‌سازی این تابع پیچیدگی خاصی ندارد. در تابع `main` نیز، ابتدا مقادیر مربوط به تعداد `goroutine`ها، `GOMAXPROCS` نوع لود کاری، و تعداد اجرای حلقه در تابع `cpuHeavyTask` (مقدار پیش‌فرض 80 میلیون است) از آرگومان‌های ورودی برنامه دریافت شده و پس از بررسی صحت آن‌ها، تابع `runExperiment` صدا زده شده و مقادیر معیارها توسط تابع `printMetrics` چاپ می‌شود. تابع `runExperiment` و تابع `main` در زیر قابل مشاهده‌اند.

```
func runExperiment(numGoroutines int, gomax int, workload WorkloadType, iterations int) Metrics {
    runtime.GOMAXPROCS(gomax)

    var wg sync.WaitGroup
    var blockingCount int64

    // time per each goroutine
    delay := make([]time.Duration, numGoroutines)

    startAll := time.Now()

    for g := 0; g < numGoroutines; g++ {
        wg.Add(1)
        go func(id int) {
            defer wg.Done()

            start := time.Now()

            switch workload {
            case CPUWorkload:
                cpuHeavyTask(iterations)
            case MixedWorkload:
                mixedTask(iterations)
                atomic.AddInt64(&blockingCount, 1)
            }

            delay[id] = time.Since(start)
        }(g)
    }

    wg.Wait()
    totalTime := time.Since(startAll)
```

```

// metrics
var minDelay time.Duration = delay[0]
var maxDelay time.Duration = delay[0]

var sum float64 = 0
for _, d := range delay {
    if d < minDelay {
        minDelay = d
    }
    if d > maxDelay {
        maxDelay = d
    }
    sum += float64(d)
}

mean := sum / float64(numGoroutines)

var variance float64 = 0
for _, d := range delay {
    diff := float64(d) - mean
    variance += diff * diff
}
variance /= float64(numGoroutines)
stdDev := math.Sqrt(variance)

throughput := float64(numGoroutines) / totalTime.Seconds()

return Metrics{
    TotalTime:    totalTime,
    Throughput:   throughput,
    MeanDelay:    time.Duration(mean),
    MinDelay:     minDelay,
    MaxDelay:     maxDelay,
    StdDevDelay:  time.Duration(stdDev),
    Goroutines:   numGoroutines,
    GOMAXPROCS:   gomax,
    NumBlocking:  blockingCount,
    Workload:     workload,
}
}

```

```

func main() {
    workloadFlag := flag.String("workload", "cpu", "Workload type: cpu or mixed")
    goroutinesFlag := flag.Int("goroutines", 8, "Number of goroutines to run")
    procsFlag := flag.Int("procs", runtime.NumCPU(), "GOMAXPROCS value")
    iterationsFlag := flag.Int("iterations", 80_000_000, "Complexity of workload computation")

    flag.Parse()

    workloadType := WorkloadType(*workloadFlag)
    if workloadType != CPUWorkload && workloadType != MixedWorkload {
        fmt.Println("Invalid workload type. Use 'cpu' or 'mixed'.")
        return
    }

    metrics := runExperiment(
        *goroutinesFlag,
        *procsFlag,
        workloadType,
        *iterationsFlag,
    )

    printMetrics(metrics)
}

```

اجرا:

برای اجرا کد می‌توان مطابق الگوی زیر عمل کرد:

```
go run main.go -workload=cpu -goroutines=8 -procs=1 -iterations=80000000
```

برای راحتی در اجرای برنامه با تنظیمات متفاوت، اسکریپت بش زیر نوشته شده است که با تکرار بر روی مقادیر مربوط به نوع لود کاری، تعداد GOMAXPROCS (1، 2 و تعداد پردازنده دستگاه که برابر با 16 است) و تعداد goroutine‌ها (1، 2، 4، 8، 16، 32، 64)، برنامه را برای هر حالت تنظیمات اجرا می‌کند و خروجی را هر بار به فایل results.txt اضافه می‌کند. تصویر این اسکریپت در زیر مشاهده می‌شود:

```
#!/bin/bash

for type in "cpu" "mixed"; do
  for g in 1 2 4 8 16 32 64; do
    for p in 1 2 $(nproc); do
      go run main.go -workload=$type -goroutines=$g -procs=$p >> results/results.txt
    done
  done
done
```

همچنین یک نمونه خروجی اجرا برای یکی از تنظیمات به صورت زیر است:

```
=====
| | | | EXPERIMENT RESULTS
=====
Workload Type      : cpu
GOMAXPROCS         : 1
Goroutines         : 1
-----
Total Runtime      : 40.380449ms
Throughput         : 24.7645 ops/sec
Mean Delay         : 40.367124ms
Min Delay          : 40.367124ms
Max Delay          : 40.367124ms
StdDev Delay       : 0s
Blocking Events    : 0
=====
```

تحلیل نتایج:

جدول شامل مقادیر معیارها برای هر کدام از حالت‌های اجرا در صفحه بعد قابل مشاهده است.

برای بررسی و تحلیل بهتر و راحت‌تر الگوهای موجود و پاسخ به سوالات مطرح شده، نمودارهای مهم مربوط به تغییر مقدار معیارها را نسبت به مقادیر مختلف متغیرهای تحت بررسی در ادامه رسم می‌کنیم و هر کدام را به اختصار توضیح می‌دهیم. مقادیر استفاده شده در نمودارها همان مقادیر موجود در جدول هستند و الگوهای استخراج شده در جدول نیز قابل مشاهده‌اند.

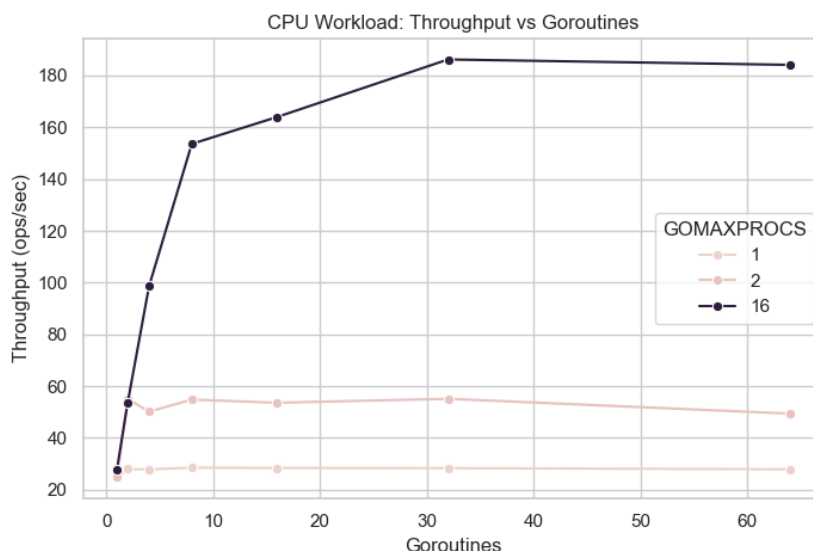
در ادامه با استفاده از نمودارها، تاثیر مقادیر مختلف GOMAXPROCS، تعداد goroutine و نوع لود کاری را بر روی معیارهایی مانند throughput، زمان اجرای کل و... مشاهده می‌کنیم.

Workload	GOMAX-PROCS	Goroutines	Blocking Events	StdDev Delay	Max Delay	Min Delay	Mean Delay	Throughput (ops/sec)	Total Runtime
CPU	1	1	0	0	0.040367	0.040367	0.040367	24.7645	0.04038
CPU	2	1	0	0	0.039962	0.039962	0.039962	25.003	0.039995
CPU	16	1	0	0	0.035708	0.035708	0.035708	27.992	0.035725
CPU	1	2	0	0.005043	0.071111	0.061025	0.066068	28.0858	0.07121
CPU	2	2	0	0.0000487	0.036291	0.036193	0.036242	55.0724	0.036316
CPU	16	2	0	0.000256	0.037254	0.036742	0.036998	53.6619	0.03727
CPU	1	4	0	0.019877	0.138776	0.087756	0.110159	27.8412	0.143672
CPU	2	4	0	0.008535	0.075482	0.053924	0.064164	50.1559	0.079751
CPU	16	4	0	0.000266	0.040429	0.039715	0.040059	98.6378	0.040552
CPU	1	8	0	0.045083	0.272255	0.140833	0.188904	28.5314	0.280393
CPU	2	8	0	0.022998	0.132598	0.056	0.101723	54.8403	0.145878
CPU	16	8	0	0.003829	0.05205	0.039819	0.045875	153.6006	0.052083
CPU	1	16	0	0.089371	0.547858	0.252552	0.350774	28.4012	0.563357
CPU	2	16	0	0.053699	0.284996	0.119207	0.202601	53.6098	0.298453
CPU	16	16	0	0.006253	0.097446	0.075323	0.082279	164.0136	0.097553
CPU	1	32	0	0.16834	1.076318	0.489146	0.665872	28.3403	1.129135
CPU	2	32	0	0.110691	0.577259	0.198367	0.374354	55.1343	0.5804
CPU	16	32	0	0.011287	0.154401	0.113395	0.134768	186.2233	0.171837
CPU	1	64	0	0.463199	1.972979	0.233317	0.938308	27.908	2.293245
CPU	2	64	0	0.06719	0.297191	0.067277	0.164536	49.4193	1.295041
CPU	16	64	0	0.04024	0.193798	0.049343	0.129861	184.1439	0.347554
mixed	1	1	1	0	0.039604	0.039604	0.039604	25.2395	0.03962
mixed	2	1	1	0	0.039299	0.039299	0.039299	25.4328	0.039319
mixed	16	1	1	0	0.040185	0.040185	0.040185	24.8643	0.040218
mixed	1	2	2	0.000145	0.039519	0.03923	0.039375	41.1759	0.048572
mixed	2	2	2	0.000363	0.039274	0.038547	0.038911	50.8862	0.039303
mixed	16	2	2	0.0004	0.049133	0.048332	0.048732	40.6894	0.049153
mixed	1	4	4	0.000155	0.044896	0.044516	0.044655	55.9492	0.071493
mixed	2	4	4	0.003859	0.04943	0.039578	0.042846	78.9809	0.050645
mixed	16	4	4	0.000509	0.040465	0.039279	0.03979	98.7647	0.0405
mixed	1	8	8	0.026571	0.105849	0.037855	0.062623	56.0281	0.142785
mixed	2	8	8	0.003559	0.056413	0.045209	0.047104	107.3912	0.074494
mixed	16	8	8	0.000846	0.042553	0.039576	0.040567	187.8824	0.04258
mixed	1	16	16	0.070171	0.229737	0.037555	0.098352	56.3102	0.28414
mixed	2	16	16	0.036712	0.154706	0.03876	0.101776	84.7283	0.188839
mixed	16	16	16	0.006399	0.077463	0.056115	0.068735	186.4681	0.085806
mixed	1	32	32	0.090469	0.330154	0.037456	0.118347	56.2968	0.568416
mixed	2	32	32	0.075648	0.249641	0.038149	0.103802	105.8937	0.30219
mixed	16	32	32	0.004121	0.071256	0.054188	0.062631	373.1279	0.085761
mixed	1	64	64	0.186027	0.620788	0.03711	0.197754	56.7644	1.127467

mixed	2	64	64	0.141961	0.481475	0.037885	0.186527	94.5568	0.676842
mixed	16	64	64	0.015665	0.122553	0.050272	0.076238	354.4462	0.180563

1) تاثیر مقادیر GOMAXPROCS و تعداد goroutine ها بر throughput:

نمودار زیر، رابطه میان تعداد goroutine ها بر throughput را برای تعداد گوناگون GOMAXPROCS در حالت لود کاری CPU-Bound نشان می‌دهد:

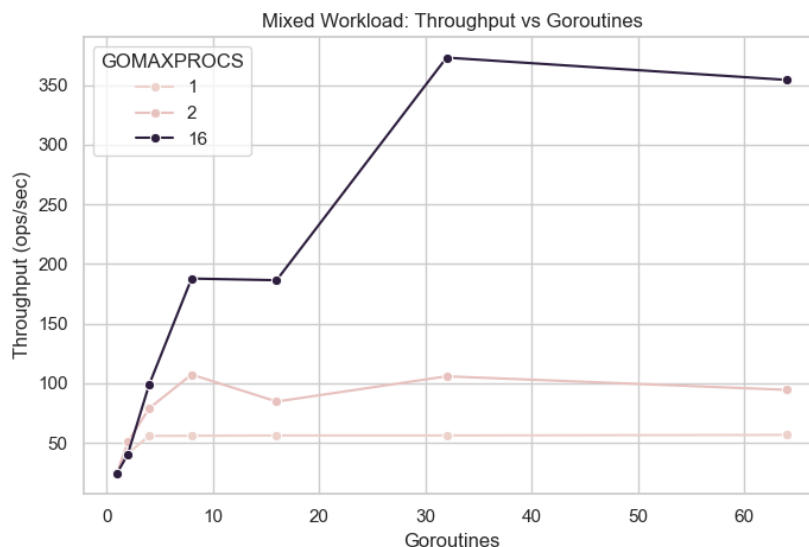


همان‌گونه که دیده می‌شود، با افزایش تعداد goroutine ها، مقدار throughput نیز به طور کلی افزایش پیدا می‌کند که این به دلیل قابلیت موازی‌سازی بیشتر و بهتر اجرا است که همان نتیجه مورد انتظار است. نکته جالب و شایان توجه این است که همان‌طور که دیده می‌شود، با افزایش تعداد روتین‌ها از یک حدی به بعد، مقدار throughput به حالت اشباع می‌رسد و دیگر افزایشی ندارد و حتی با افزایش بیشتر تعداد روتین‌ها، کاهش جزئی‌ای نیز در مقدار throughput دیده می‌شود؛ به عبارت دیگر، با افزایش تعداد روتین‌ها از حدی به بعد، دیگر به اندازه افزایش تعداد روتین‌ها، نمی‌توان موازی‌سازی برنامه را بهبود داد، بلکه هزینه scheduling بالاتر از حد معقول می‌رود و همین امر سبب ثابت ماندن و یا حتی کاهش جزئی مقدار throughput می‌شود. مقدار این حد اشباع برای تعداد روتین‌ها در هر سه حالت GOMAXPROCS جایی بین حالت 32 و 64 بوده است.

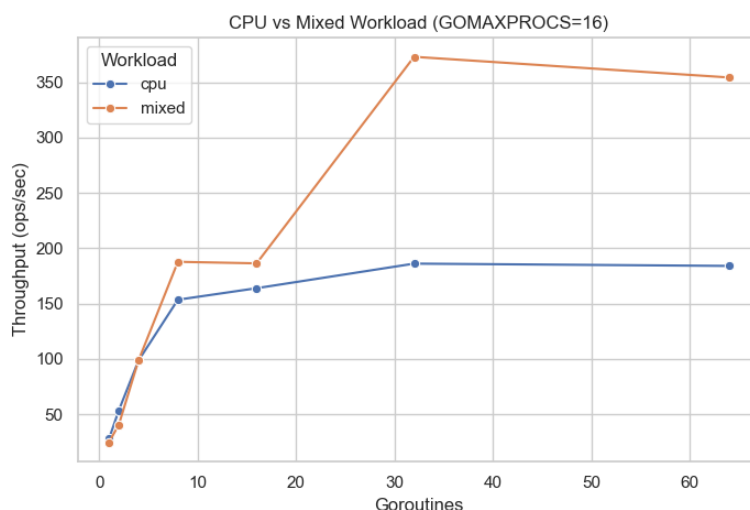
همچنین، الگوی واضح دیگری که در نمودار دیده می‌شود، اثر واضح تعداد GOMAXPROCS بر throughput برنامه است به طوری که با افزایش این مقدار از 1 به 2 و از 2 به 16، مقدار throughput به طرز قابل توجهی افزایش می‌یابد که طبیعتاً در نتیجه افزایش قابلیت اجرای موازی روتین‌ها است.

نکته دیگر این است که همان‌طور که دیده می‌شود، در حالتی که مقدار GOMAXPROCS برابر با 1 بوده است، ما عملاً مقیاس‌پذیری خاصی نداشته‌ایم و با افزایش تعداد روتین‌ها، throughput افزایش خاصی نکرده است، که خوب به این دلیل است که به طور منطقی ما تنها یک پردازنده یا هسته پردازشی داریم و در نتیجه اجرای موازی رخ نمی‌دهد، بلکه با افزایش تعداد روتین‌ها صرفاً تعداد context-switch افزایش می‌یابد.

شکل زیر همان نمودار بالا را برای حالت لود کاری ترکیبی نشان می‌دهد:



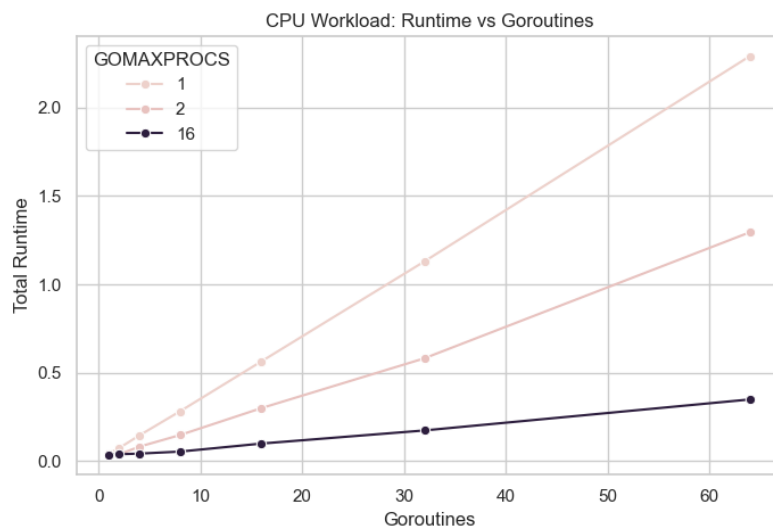
همان الگوهایی که در حالت لود کاری CPU-Bound وجود داشت و شرح داده شد، در این نمودار نیز مشاهده می‌شود، هر چند که روند تغییر مقدار throughput و خود مقدار آن در دو نمودار متفاوت است. همچنین حد اشباع برای تعداد روتین‌ها در این نمودار نیز مانند نمودار لود کاری CPU-Bound است. برای مقایسه بهتر تغییرات throughput در دو لود کاری، نمودار زیر را در نظر می‌گیریم:



می‌توان مشاهده کرد که در اغلب نقاط، throughput برای حالت لود کاری ترکیبی از لود کاری CPU-Bound بیشتر بوده است. یکی از دلایل آن این است که در لود کاری ترکیبی، روتین‌ها شامل عملیات sleep می‌شوند که این فرصت را به سایر روتین‌ها می‌دهد که تا رسیدن به قسمت sleep خود، محاسبات خود را اجرا کنند و سپس با رسیدن به sleep، مجدداً این فرآیند تکرار می‌شود؛ در نتیجه، در نهایت به نوعی scheduler تاخیر ناشی از بلاک کردن روتین‌ها را با اجرای سایر روتین‌ها جبران می‌کند و در یک زمان، عملیات روتین‌های بیشتری انجام شده و throughput افزایش می‌یابد. همچنین روند تغییرات مقدار throughput در حالت لود کاری ترکیبی تا حدی متفاوت از لود کاری نوع دیگر است و دو نمودار در اکثر بازه‌ها، شیب‌های متفاوتی را دارا هستند.

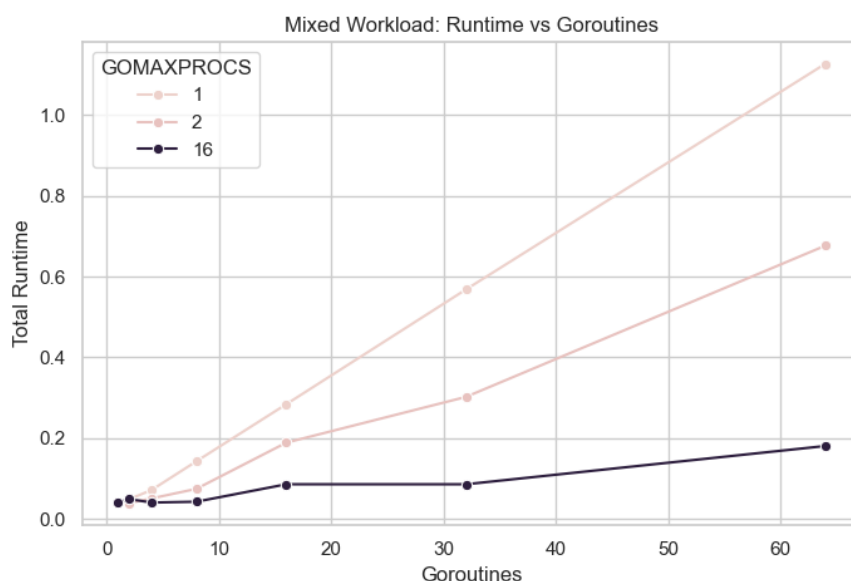
(2) تاثیر مقادیر GOMAXPROCS و تعداد goroutine بر runtime:

نمودار زیر، رابطه میان زمان اجرا با مقادیر GOMAXPROCS و تعداد روتین‌ها را در لود کاری محاسباتی نشان می‌دهد:



با توجه به نمودار، با افزایش تعداد روتین‌ها، به طور کلی زمان اجرای برنامه افزایش پیدا می‌کند که البته روند افزایش زمان اجرا، با افزایش تعداد هسته پردازشی کاهش می‌یابد که این به دلیل قابلیت موازی‌سازی با افزایش تعداد هسته‌ها است که با اجرای موازی تعداد بیشتری از روتین‌ها، زمان کل اجرای برنامه نسبت به حالتی که هسته‌های پردازشی کمتری برای پردازش موازی در اختیار داریم، کمتر می‌باشد.

نمودار صفحه بعد نیز همین رابطه را برای لود کاری ترکیبی نشان می‌دهد که در آن نیز همین الگو برقرار است. همچنین از مقایسه زمان اجرای حالت لود کاری CPU-Bound با حالت لود کاری ترکیبی، متوجه می‌شویم که در موارد تنظیمات مشابه، زمان اجرای حالت لود کاری CPU-Bound بیشتر بوده است که این احتمالاً به دلیل پردازش محسباتی بیشتر هر روتین در هر بار اجرا بر روی یک هسته می‌باشد. همچنین تغییراتی جزئی در روند تغییرات افزایش زمان کل اجرا مشاهده می‌شود، هر چند که روند کلی تغییرات تا حدود زیادی میان دو نمودار مشابه است.



(3) دیگر الگوها:

اصلی‌ترین الگوهای موجود در دو قسمت قبلی به تفصیل شرح داده شد. می‌توان از جدول، الگوهای دیگری نیز استخراج کرد؛ مانند اینکه تعداد دفعات بلاک شدن برنامه تنها در لود کاری ترکیبی با افزایش تعداد روتین‌ها افزایش یافته و با آن رابطه خطی دارد و یا اینکه به طور کلی، در هر دو نوع کاری، با افزایش تعداد روتین‌ها، انحراف معیار زمان اجرای روتین‌ها نیز افزایش یافته است که هر دو مورد از نتیجه‌های بدیهی و قابل انتظار می‌باشند.

- (1) با افزایش تعداد goroutine ها چه اتفاقی برای زمان اجرا می افتد؟ همان طور که در بخش دوم تحلیل نشان داده شد، با افزایش تعداد روتین ها، زمان اجرا نیز افزایش می یابد، اما هر چه مقدار GOMAXPROCS بیشتر باشد، به علت قابلیت موازی سازی بیشتر، این نرخ افزایش کمتر است.
- (2) آیا افزایش تعداد goroutine ها همیشه باعث افزایش throughput می شود؟ خیر، همان طور که اشاره شد، با افزایش تعداد روتین ها تا حدی، مقدار throughput نیز افزایش می یابد، اما با گذر از آن حد که آن را حد اشباع تعداد روتین ها نامیدیم، throughput افزایش چندانی پیدا نمی کند و حتی بعضا کاهش نیز می یابد.
- (3) تفاوت رفتار برنامه در $GOMAXPROCS=1$ و $GOMAXPROCS=NumCPU$ چیست؟ همان طور که دیده توضیح داده شد، در حالت $GOMAXPROCS$ برابر با 1، ما عملا هیچ موازی سازی ای نخواهیم داشت و با افزایش تعداد روتین ها، صرفا تعداد context-switch مورد نیاز و زمان اجرای برنامه افزایش یافته بدون اینکه throughput افزایش خاصی داشته باشد؛ اما با افزایش تعداد $GOMAXPROCS$ و رساندن آن به مقدار $NumCPU$ که ای مقدار برای ما 16 بود، قابلیت اجرای موازی افزایش پیدا می کند و در اثر آن، مقدار throughput نیز افزایش چشمگیری می یابد و همچنین میزان افزایش زمان اجرا نیز کاهش می یابد و مقدار آن نسبت به حالتی که فقط یک هسته پردازشی داشتیم، کمتر می باشد.
- (4) در کدام نوع workload اثرات زمانبندی و switching بیشتر دیده می شود؟ در لود کاری ترکیبی، چرا که در این لود کاری، روتین ها بعد از مدتی به sleep می روند و در این حال، زمان بند باید مجددا زمان بندی کند و یک context-switch رخ دهد، در حالی که در لود کاری محاسباتی، تا تمام نشدن محاسبات یک روتین به صورت کامل، این اعمال انجام نخواهند شد و در نتیجه نرخ زمان بندی و context-switch کردن نسبت به حالت ترکیبی کمتر است. این موضوع را می توان با استناد به تعداد دفعات بلاک کردن و همچنین انحراف معیار زمان اجرا نیز متوجه شد.
- (5) از چه نقطه ای به بعد، افزایش همزمانی مفید نیست یا حتی کارایی را کاهش می دهد؟ این همان مقدار اشباع تعداد روتین ها است که همان طور که در نمودارها دیده شد، جایی بین 32 تا 64 روتین است.

بخش سوم: ساخت سرویس، کانتینرسازی و اجرا در ماشین مجازی

در این قسمت، یک سرویس HTTP با استفاده از زبان Go پیاده سازی شد و سپس با استفاده از Docker کانتینر سازی گردید تا اجرای آن در محیط های مختلف ساده و قابل بازتولید باشد. هدف از این بخش ایجاد یک سرویس محاسباتی ساده است که بتواند درخواست های HTTP را دریافت کرده و عملیات های حسابی پایه را انجام دهد. علاوه بر پیاده سازی سرویس، این بخش شامل ساخت Image داکر، اجرای کانتینر در محیط لینوکس و بررسی عملکرد صحیح سرویس از طریق درخواست های HTTP نیز می شود.

طراحی این بخش به گونه ای انجام شده است که ساختار سرویس تا حد امکان ساده، قابل فهم و در عین حال قابل گسترش باشد. هسته اصلی برنامه یک سرور HTTP است که با استفاده از کتابخانه استاندارد net/http در زبان Go پیاده سازی شده است. این سرور دو endpoint اصلی در اختیار کاربر قرار می دهد. endpoint اول مسیر health/ است که برای بررسی وضعیت سرویس استفاده می شود و در صورت فعال بودن سرویس، یک پاسخ ساده با وضعیت ok برمی گرداند. همچنین لازم به ذکر است که این endpoint معمولاً در سیستم های توزیع شده برای بررسی سلامت سرویس ها مورد استفاده قرار می گیرد.

endpoint دوم مسیر compute/ است که برای انجام عملیات های محاسباتی طراحی شده است. در این endpoint کاربر می تواند از طریق پارامترهای query نوع عملیات و مقادیر ورودی را مشخص کند. پارامتر op نوع عملیات را مشخص می کند و پارامترهای a و b نیز مقادیر عددی مورد نیاز برای انجام محاسبه هستند. سرویس پس از دریافت درخواست، ابتدا اعتبار پارامترها را بررسی می کند. در صورتی که پارامترها ناقص باشند یا مقادیر عددی معتبر نباشند، سرویس یک پیام خطا به همراه کد وضعیت مناسب HTTP باز می گرداند. در صورتی که عملیات معتبر باشد، نتیجه محاسبه انجام شده و در قالب JSON به کاربر ارسال می شود. این طراحی باعث می شود سرویس رفتار قابل پیش بینی داشته باشد و خطاها نیز به صورت شفاف مدیریت شوند.

در پیاده سازی این بخش تلاش شده است مدیریت خطا به صورت کامل انجام شود. برای مثال اگر کاربر عملیاتی غیرمجاز ارسال کند، اگر تعداد پارامترها کافی نباشد، اگر ورودی ها عددی نباشند یا اگر عملیات تقسیم بر صفر رخ دهد، سرویس پیام خطای مناسب را برمی گرداند. همچنین متدهای HTTP نیز کنترل شده اند تا تنها درخواست های GET برای این endpoint ها پذیرفته شوند.

پس از پیاده سازی سرویس، مرحله بعدی کانتینرسازی برنامه با استفاده از Docker انجام شد. برای این کار یک Dockerfile طراحی شد که از روش multi-stage build استفاده می کند. در مرحله اول از یک image مبتنی بر Go برای کامپایل برنامه استفاده می شود. در این مرحله کد منبع برنامه به یک فایل اجرایی تبدیل می شود. سپس در مرحله دوم تنها فایل اجرایی تولید شده در یک image سبک تر مبتنی بر Alpine Linux کپی می شود. این روش باعث می شود image نهایی بسیار کوچک تر و امن تر باشد زیرا ابزار های کامپایل و وابستگی های اضافی در آن وجود ندارند.

برای ساخت image داکر، دستور docker build در پوشه پروژه اجرا شد. این دستور با استفاده از Dockerfile موجود در پروژه، مراحل build را اجرا کرده و image نهایی با نام go-service ساخته می شود که در ادامه اسکرین شات اجرای دستور در ترمینال لینوکس را آورده ام.

```

navid@navid-VirtualBox:~/navid/8th_term/Dist/HW1/part3$ docker build -t go-service .
Sending build context to Docker daemon 129.5MB
Step 1/10 : FROM golang:1.22-alpine AS builder
1.22-alpine: Pulling from library/golang
1f3e46996e29: Pull complete
4d75fd4b7386: Pull complete
afa154b433c7: Pull complete
5f837c998576: Pull complete
4f4fb700ef54: Pull complete
Digest: sha256:1699c10032ca2582ec89a24a1312d986a3f094aed3d5c1147b19880afe40e052
Status: Downloaded newer image for golang:1.22-alpine
--> 4129f51f28c9
Step 2/10 : ENV GOPROXY=https://mirror-go.runflare.com
--> Running in cc349b84c5ab
Removing intermediate container cc349b84c5ab
--> 5be5c942d746
Step 3/10 : WORKDIR /app
--> Running in b5ab5b57c7bb
Removing intermediate container b5ab5b57c7bb
--> 3e7c8bf5b063
Step 4/10 : COPY . .
--> 20b38a10c5b2
Step 5/10 : RUN go build -o service main.go
--> Running in 913a9a4a0322
Removing intermediate container 913a9a4a0322
--> 662f0ae5db3c
Step 6/10 : FROM alpine:latest
latest: Pulling from library/alpine
6a0ac1617861: Pull complete

```

```

Digest: sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
Status: Downloaded newer image for alpine:latest
--> 3cb067eab609
Step 7/10 : WORKDIR /app
--> Running in 3b48537bf31a
Removing intermediate container 3b48537bf31a
--> 58e636dcf678
Step 8/10 : COPY --from=builder /app/service .
--> cac627cbb087
Step 9/10 : EXPOSE 8080
--> Running in cbf3a6f09b45
Removing intermediate container cbf3a6f09b45
--> 384d03456e79
Step 10/10 : CMD ["/service"]
--> Running in d46ea6e6614f
Removing intermediate container d46ea6e6614f
--> e814e5d6feee
Successfully built e814e5d6feee
Successfully tagged go-service:latest

```

پس از ساخته شدن **image**، سرویس با استفاده از دستور **docker run** اجرا شد. در این دستور پورت **8080** کانتینر به پورت **8080** ماشین میزبان متصل شد تا بتوان از خارج کانتینر به سرویس دسترسی داشت. با اجرای این دستور، کانتینر اجرا شده و سرور **HTTP** شروع به گوش دادن روی پورت **8080** می‌کند.

```

navid@navid-VirtualBox:~/navid/8th_term/Dist/HW1/part3$ docker run -p 8080:8080 go-service
Server is running on port 8080...

```

برای بررسی عملکرد سرویس، در یک ترمینال دیگر در خواست‌های مختلفی با استفاده از ابزار **curl** ارسال شد. ابتدا درخواست های صحیح برای انجام عملیات‌های محاسباتی ارسال شدند تا اطمینان حاصل شود که سرویس به درستی عملیات‌ها را پردازش می‌کند و نتیجه را در قالب **JSON** باز می‌گرداند.

```
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW1/part3
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW1/part3$ docker run -p 8080:8080 go-service
Server is running on port 8080...

navid@navid-VirtualBox:~/navid/8th_term/Dist/HW1/part3$ curl http://localhost:8080/health
{"status":"ok"}
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW1/part3$ curl "http://localhost:8080/compute?op=add&a=5&b=7"
{"operation":"add","a":5,"b":7,"result":12}
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW1/part3$
```

در مرحله بعد، چندین درخواست نادرست نیز ارسال شد تا مکانیزم مدیریت خطا در سرویس بررسی شود. این درخواست ها شامل مواردی است که توضیح داده شد؛ مواردی از جمله ارسال عملیات نامعتبر، ارسال پارامتر ناقص، وارد کردن مقدار غیر عددی و همچنین تقسیم بر صفر بودند. سرویس در تمام این موارد پیام خطای مناسب را تولید کرد و از بروز خطا یا توقف برنامه جلوگیری شد.

```
navid@navid-VirtualBox: ~/navid/8th_term/Dist/HW1/part3
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW1/part3$ docker run -p 8080:8080 go-service
Server is running on port 8080...

navid@navid-VirtualBox:~/navid/8th_term/Dist/HW1/part3$ curl "http://localhost:8080/compute?op=div&a=10&b=0"
{"error":"division by zero"}
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW1/part3$ curl "http://localhost:8080/compute?op=test&a=1&b=2"
{"error":"invalid operation: test"}
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW1/part3$ curl "http://localhost:8080/compute?op=add&a=5"
{"error":"missing parameters: op, a, b are required"}
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW1/part3$ curl "http://localhost:8080/compute?op=add&a=hello&b=5"
{"error":"parameter a must be numeric"}
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW1/part3$ curl "http://localhost:8080/compute?op=add&a=5&b=xyz"
{"error":"parameter b must be numeric"}
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW1/part3$ curl -X POST http://localhost:8080/health
{"error":"method not allowed"}
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW1/part3$ curl -X POST "http://localhost:8080/compute?op=add&a=1&b=2"
{"error":"method not allowed"}
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW1/part3$
```

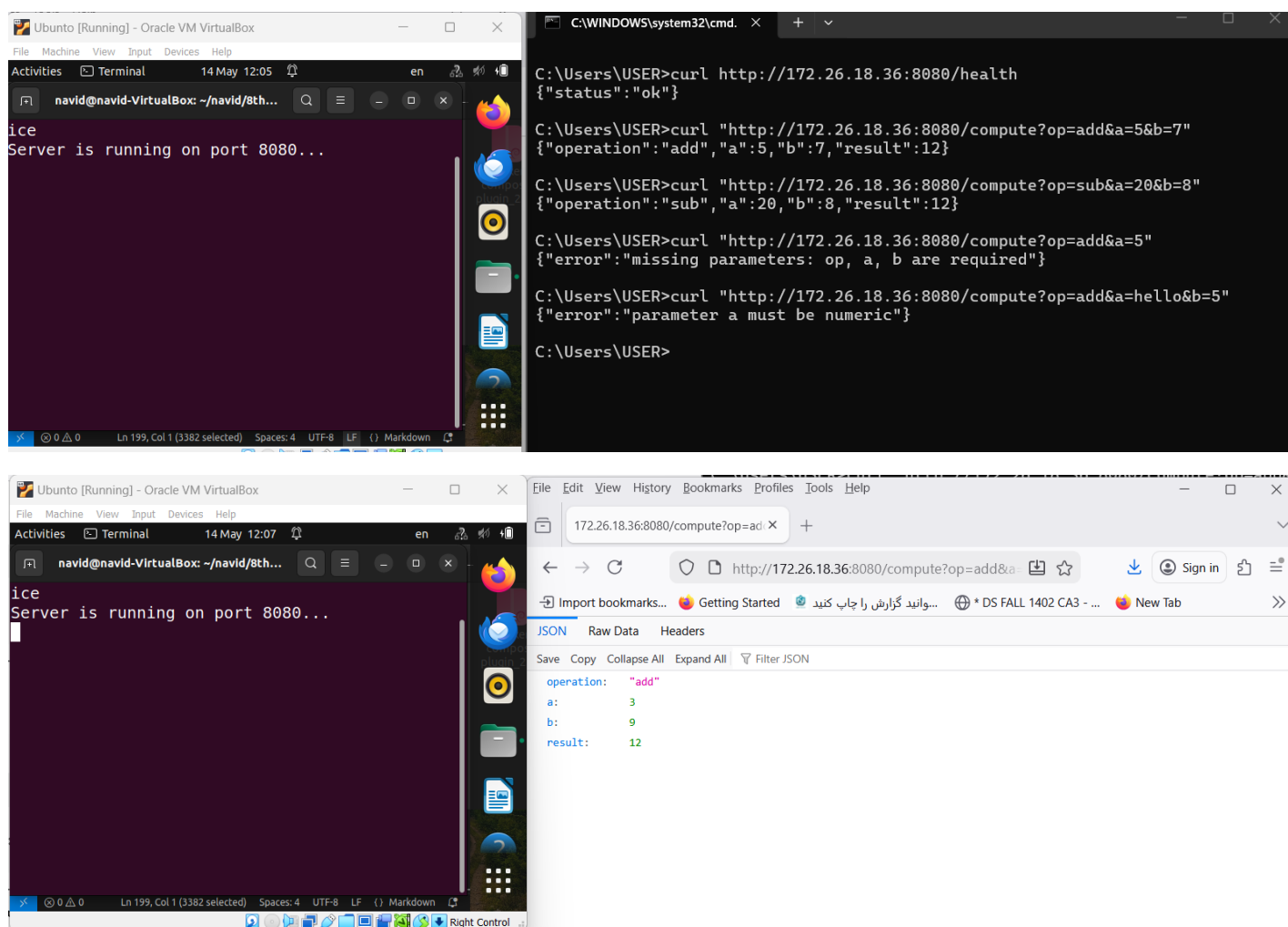
علاوه بر استفاده از ابزار `curl` در ترمینال، عملکرد سرویس از طریق مرورگر نیز بررسی شد. در حالی که کانتینر در حال اجرا بود، یک درخواست HTTP از طریق مرورگر Firefox به endpoint سرویس ارسال شد و پاسخ JSON به درستی دریافت گردید. این تست در واقع برای این بود که نشان دهیم سرویس علاوه بر ترمینال، از طریق مرورگر نیز قابل دسترسی است و ارتباط شبکه به درستی برقرار شده است.

```
navid@navid-VirtualBox:~/navid/8th_term/Dist/HW1/part3$ docker run -p 8080:8080 go-service
Server is running on port 8080...

localhost:8080/compute?op=div&a=10&b=0
http://localhost:8080/compute?op=div&a=10&b=0
JSON Raw Data Headers
Save Copy Collapse All Expand All Filter JSON
error: "division by zero"
```

در تصاویر مشاهده می شود که درخواست های `curl` و همچنین درخواست ارسال شده از مرورگر Firefox از همان VM به خود ماشین مجازی ارسال شدند که نشان دهنده برقراری ارتباط موفق Guest با Guest است.

اما در ادامه تصاویری آمده اند که نشان دهنده برقراری ارتباط موفق از Host به Guest می باشند؛ در تصویر اول درخواست های curl از طریق windows power shell و در تصویر دوم درخواست از طریق مرورگر Firefox از Host به ماشین مجازی (Guest) ارسال شدند.



در مجموع، در این بخش یک سرویس HTTP ساده پیاده سازی شد که می تواند درخواست های محاسباتی را پردازش کند. سپس این سرویس با استفاده از Docker کانتینر سازی شد تا اجرای آن در محیط های مختلف ساده تر شود. در نهایت سرویس در یک ماشین مجازی لینوکس اجرا شد و با استفاده از ابزار های مختلف مورد آزمایش قرار گرفت. تمام مراحل build و run داکر داخل ماشین مجازی لینوکس انجام شده و هیچ بخشی از اجرای سرویس روی سیستم Host صورت نگرفته است. همچنین در اسکرین شات های قبلی هم اتصال Guest -> Guest بررسی شد و هم اتصال Guest -> Host نتایج نشان داد که سرویس به درستی اجرا می شود، درخواست ها را پردازش می کند و در مواجهه با خطاها نیز رفتار قابل پیش بینی و پایداری دارد.